

Pętle

1. Pętla realizowana instrukcją skoku

Konstrukcja algorytmu typu **pętla** to fragmenty algorytmu wykonujące się kilkakrotnie, dopóki spełnione są określone warunki. W pętli możemy wyróżnić (najczęściej) podstawowe elementy:

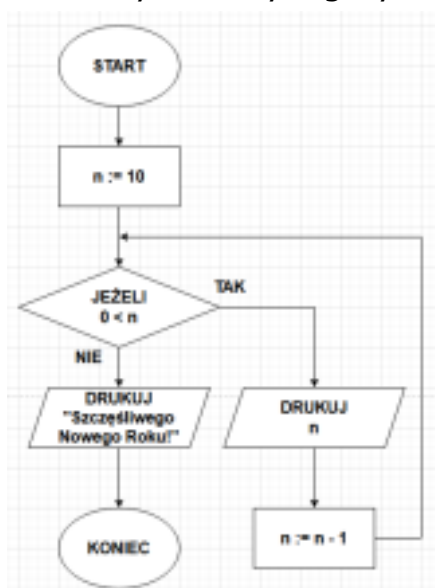
- licznik (w jakiejś formie) oraz nadanie mu wartości początkowej.
- warunek wykonania/stopu. Nazwa zależy od konstrukcji warunku. Jeśli warunek jest tak skonstruowany, że „prawda” powoduje kolejne wykonanie, natomiast „fałsz” przerwanie takiej pętli, to nazywamy go warunkiem wykonania. Jeśli warunek jest tak skonstruowany, że „fałsz” powoduje kolejne wykonanie, natomiast „prawda” przerwanie takiej pętli, to nazywamy go warunkiem stopu.
- modyfikator licznika (w jakiejś formie),

Czasami spotyka się pętle bez użycia licznika, z samym tylko warunkiem stopu/wykonania.

Uwaga! W profesjonalnym programowaniu używanie instrukcji **goto** jest odradzane ze względu na to, że takie przeskoki w dowolne miejsca programu mogą powodować, że będziemy m.in. gubić zmienne, a tym samym powodować wycieki pamięci. Na przykład, przejście kodu przez `}` usuwa z pamięci zmienne zadeklarowane w tym bloku kodu. Wyskoczenie z bloku kodu poprzez **goto** pominię ten krok (zmienne, chociaż już niepotrzebne, zostaną w pamięci).

Warto jednak wiedzieć, że taka instrukcja jest oraz w jaki sposób działa.

Przykładowy algorytm licznika odliczający sekundy do Nowego Roku:



```
int n = 10;
count:

if (0 < n)
{
    cout << n << endl;
    n--;
    goto count;
}

cout << "Szczęśliwego Nowego Roku!\n";
```

2. Postinkrementacja vs preinkrementacja

Inkrementacją nazywamy zwiększenie wartości zmiennej o jeden. Zazwyczaj występuje (dla wygody zapisu kodu) w połączeniu z innym współzależnym kodem. Możemy zdecydować, czy dokonujemy jej **przed wykonaniem** czy dopiero **po wykonaniu** współzależnego kodu. Wykonanie

„przed” nazywa się **preinkrementacją**, wykonanie „po” nazywa się **postinkrementacją**.

<pre>int n = 10; cout << n++ << endl; cout << n++ << endl; cout << n++ << endl; /* Postinkrementacja, drukujemy kolejno: 10 11 12 końcowa wartość "n" to 13 */</pre>	<pre>int n = 10; cout << ++n << endl; cout << ++n << endl; cout << ++n << endl; /* Preinkrementacja, drukujemy kolejno: 11 12 13 końcowa wartość "n" to 13 */</pre>
---	--

Uwaga na wielokrotne używanie inkrementacji/dekrementacji w wersjach pre/post w niektórych funkcjach. Przykładowo zapisany poniższy kod wyświetli na ekranie kolejno wartości: **12 11 10**.

```
int n = 10;
cout << n++ << " " << n++ << " " << n++ << endl;
```

Z kolei poniższy kod wyświetli na ekranie trzy dokładnie takie same wartości: **13 13 13** (preinkrementacja zadziała przed jakimkolwiek wydrukiem). *int n = 10;*

```
cout << ++n << " " << ++n << " " << ++n << endl;
```

Jak widać nie do końca działa to tak, jakby się wydawało, że działać powinno. :)

Postdekrementacja vs predekrementacja

Analogicznie działa dekrementacja zmiennej, czyli zmniejszenie jej o jeden. W zależności od zastosowania zapisu „**pre**” lub „**post**”, wartość zmiennej będzie zmniejszana **przed wykonaniem** lub **po wykonaniu** współzależnego kodu.

3. Pętla for

Pętla **for()** ma wewnątrz 3 parametry. Są one dobrowolne, nie muszą być zadeklarowane, jeśli nie ma takiej potrzeby.

Nawet, gdy nie deklarujemy atrybutów wewnątrz, to oba średniki w nawiasie jednak muszą pozostać.

„Pusta” pętla **for()** będzie się wykonywać w nieskończoność!

```
for( ; ; )
{
    cout << "Infinito loop!\n";
}
```

Pierwszy atrybut – pomocniczy, tutaj deklarujemy zmienne, które wykorzystamy wewnątrz pętli np. zmienna odpowiedzialna za licznik.
Środkowy atrybut – (najważniejszy) instrukcja warunkowa, która steruje czy

pętla ma się wykonać, czy też nie. Dopóki warunek będzie spełniony, to pętla będzie się wykonywać. Jeśli warunek nie będzie spełniony następuje zaprzestanie wykonywania pętli.

Trzeci atrybut, pomocniczy, drobne operacje dodatkowe. Na przykład zwiększanie lub zmniejszanie zmiennej odpowiedzialnej za zliczanie wykonania pętli.

Po instrukcji **for()**, tuż po nawiasach z atrybutami, nie stawiamy średnika. Postawienie średnika sugerowałoby, że tutaj kończy się blok kodu przypisany do pętli (a nie na klamrze zamykającej).

Za pętlą powinny znaleźć się klamry zawierające blok kodu jaki ma się wykonać w tej pętli.

Obowiązują te same zasady odnośnie formatowania kodu, co dla instrukcji warunkowych. Tabowanie kodu wewnątrz bloku kodu dla zwiększenia czytelności. Stosowanie zapisu klamrowego, nawet gdy mamy do zrealizowania tylko jedną instrukcję.

4. "Pusta" pętla for() vs „pełna” pętla for()

Poniższe kody wyświetlają kolejne liczby od 1 do 100 włącznie. Wyraźnie widać wyższość „pełnej” pętli for() nad pętlą „pustą”. W „pełnej” pętli niektóre operacje wykonują się automatycznie, w pętli „pustej” zawsze musimy o nich pamiętać i zapisywać je ręcznie.

```
int i = 1;
```

```
for(;;)
```

```
{  
    if(100 < i)
```

```
{
```

```
    goto after_loop;
```

```
}
```

```
    cout << i << "\n";
```

```
    i++;
```

```
}
```

```
after_loop:
```

Zmienna pomocnicza **i** istnieje po zakończeniu pętli (niepotrzebnie zajmuje pamięć i nazwy zmiennej takiej samej już użyć nie możemy).

Modyfikacja licznika **i++** musi być zapisana zawsze na samym końcu, tuż przed klamrą zamykającą.

Punkt kontrolny musi być zadeklarowany tuż za pętlą. Wewnątrz pętli, aby ją przerwać, musi być instrukcja skoku do punktu kontrolnego **poza pętlą**. Aby się nic nie pomyliło, każda pętla musi mieć inne nazwy punktów kontrolnych.

Pełny zapis instrukcji warunkowej wyłącznie z **if()**. W kodzie warunku wyskok z kodu pętli.

```
for(int i = 1; i <= 100; i++) { cout << i << "\n";  
}
```

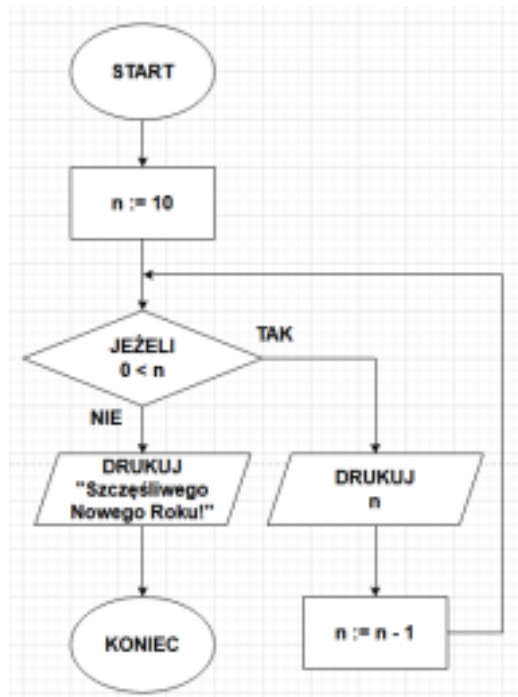
Zmienna pomocnicza **i** jest zadeklarowana w pierwszym atrybucie funkcji **for()**. Istnieje tylko na czas trwania pętli. Po jej zakończeniu jest usuwana z pamięci.

Modyfikacja licznika **i++** jest zapisana w 3 argumencie instrukcji **for()**.

Nie ma punktów kontrolnych ani skoków (zadeklarowanych w sposób jawny, one tam występują, ale są ukryte przed wzrokiem programisty).

Tylko sam warunek, zapisany w środkowym argumencie funkcji **for()**. Nie potrzeba instrukcji wyskoku z pętli.

5. Pętla for vs skoki z użyciem goto



(obraz wklejony jeszcze raz, bo łatwiej będzie analizować kody)

Algorytmy realizujące ten sam kod, zapisane z użyciem **goto** oraz **for()**

<pre> int n = 10; counter: if(0 < n) { cout << n << endl; n--; goto counter; } cout << "Szczęśliwego Nowego Roku!"; </pre>	<pre> for(int n = 10; 0 < n; n--) { cout << n << endl; } cout << "Szczęśliwego Nowego Roku!"; </pre>
---	--

6. Instrukcje sterujące: return, break, continue

Wewnątrz bloku realizowanego w pętli możemy korzystać z instrukcji sterujących, tym samym wpływając na wykonanie takiej pętli.

return – całkowicie kończy funkcję nadrzędną, a w przypadku, gdy jest użyte w main(), to całkowicie kończy program. Za instrukcją **return** powinna znaleźć się wartość właściwa dla typu funkcji (np. dla main() musi to być liczba całkowita, bo main() jest typu **int**). Jeśli funkcja jest typu void, to return jest bez atrybutu.

break – całkowicie przerywa wykonanie pętli. Przechodzi do realizacji kodu zapisanego bezpośrednio za pętlą.

continue – ignoruje dalszy kod zapisany poniżej tej instrukcji, przechodzi

od razu do kolejnego wykonania pętli.

Przykład. Poniższy kod (teoretycznie) drukuje kolejne liczby, od wartości wprowadzonej do wartości 100, a kiedy to zrobi, to drukuje końcowy komunikat „After loop!”. Jednakże:

- żadna liczba podzielna przez 10 nie zostanie wydrukowana – instrukcja **continue** powoduje zakończenie aktualnej pętli i od razu przechodzi do wykonania kolejnej.
- jeśli liczba okaże się podzielna przez 33, to pętla kończy się (instrukcja **break** powoduje przerwanie pętli), nie sprawdza więcej liczb i od razu drukuje komunikat „After loop!”.
- jeśli liczba okaże się podzielna przez 55, to niczego więcej już nie wykonamy, program ulega zakończeniu (instrukcja **return** całkowicie kończy funkcję main() - zwracamy 1, jako informację, że coś poszło nie tak, skoro program zakończył się przed czasem).

```
int main()
{
    int x;

    cin >> x;

    for( int i = x; i <= 100; i++ )
    {
        if( i % 10 == 0 )
        {
            continue;
        }

        cout << i << endl; // tutaj wydruk kolejnych liczb

        if( i % 33 == 0 )
        {
            break;
        }

        if( i % 55 == 0 )
        {
            return 1;
        }
    }

    cout << "After loop!";

    return 0;
}
```

7. Pętla while() oraz do-while()

Pętla **while()** to pętla z tylko jednym atrybutem. Wewnątrz określony jest jedynie warunek wykonywalności pętli. Pętla wykona się tak długo, jak długo będzie spełniony warunek nią sterujący.

Po instrukcji **while()**, tuż po nawiasie z warunkiem, nie stawiamy średnika.

Pętla **while()** jest szczególnie użyteczna do prostych liczników, gdzie

istnieje opcja, że kod w pętli **może się nie wykonać ani razu**. W przykładzie pętla będzie wyświetlać kolejne wartości liczbowe od liczby 9 do 0 włącznie. Warunek wykonania nie został określony w kategoriach **true/false**, pętla będzie wykonywała się tak długo, dopóki zmienna „n” będzie różna od ZERO.

```
int n = 10;

while( n-- )
{
    cout << n << endl;
}
```

Pętla **do-while()** działa podobnie jak zwykła pętla **while()**. Jedyną różnicą to taka, że warunek jest sprawdzany dopiero na samym końcu. Z tego względu pętla **do-while()** **wykona się zawsze co najmniej jeden raz!**

Po instrukcji **do** nie stawiamy średnika, natomiast po instrukcji **while()** już tak, gdyż tutaj instrukcja **while()** ma charakter kończący pętlę.

Pętla **while()** jest szczególnie użyteczna do prostych liczników, gdzie **musimy** wykonać kod **co najmniej raz**.

W przykładzie pętla będzie wyświetlać kolejne wartości liczbowe od liczby 10 do 0 włącznie. Warunek wykonania nie został określony w kategoriach **true/false**, pętla będzie wykonywała się tak długo, dopóki zmienna „n” będzie różna od ZERO.

```
int n = 10;

do
{
    cout << n << endl;
} while( n-- );
```

Teoretycznie dowolny kod realizujący jakąś pętlę możemy zapisać **zamiennie** z użyciem **for()** lub **while()** lub **do-while()**. Wybór właściwego zapisu jest determinowany tylko wygodą programisty (i czytelnością kodu).